

Integration Testing with @SpringBootTest

Interview Deep Dive — Spring Context, Test Slices, and Test Boundaries
September 2026

Integration Testing with @SpringBootTest

1. What is Integration Testing?
 2. What does @SpringBootTest do?
 3. Unit Test vs @SpringBootTest
 4. Simple Example
 5. What exactly is being tested?
 6. @SpringBootTest starts the entire application?
 7. webEnvironment
 8. @SpringBootTest + MockMvc
 9. @WebMvcTest vs @SpringBootTest
 10. @SpringBootTest + Real Database
 11. But don't use your production database
 12. @Transactional in Integration Tests
 13. Testing the Full HTTP Flow
 14. Do we mock anything in @SpringBootTest ?
 15. @MockBean / @MockitoBean
 16. Integration Test Architecture
 17. @SpringBootTest is expensive
 18. Spring Context Caching
 19. Profiles for Integration Testing
 20. What should you test with @SpringBootTest ?
 21. What NOT to do
 22. Interview Question: Is @SpringBootTest an End-to-End Test?
 23. Interview Question: Why Not Use @SpringBootTest Everywhere?
 24. @SpringBootTest vs @DataJpaTest
 25. Unit → Integration → E2E
 26. BPS-specific application
- Interview Cheat Sheet

Integration Testing with `@SpringBootTest`

For a **3+ year Spring Boot backend interview**, you should understand `@SpringBootTest` beyond simply knowing that it “starts the application.” The important part is understanding **what gets loaded, what is actually tested, how it differs from unit testing, and how you control external dependencies.**

1. What is Integration Testing?

A **unit test** tests one class in isolation.

An **integration test** verifies that **multiple components work together correctly.**

For example:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

A unit test might test only:

```
PaymentService
  ↓
Mock PaymentRepository
```

An integration test can test:

```
HTTP Request
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

The purpose is to catch problems that unit tests can miss:

- Incorrect Spring dependency injection
- Incorrect bean configuration
- Incorrect entity/repository configuration
- Transaction problems
- Serialization/deserialization problems
- Incorrect application properties
- Incorrect controller → service → repository wiring

2. What does `@SpringBootTest` do?

```
@SpringBootTest
class PaymentServiceIntegrationTest {

    @Test
    void testPayment() {
```

```
}  
}
```

`@SpringBootTest` tells Spring Boot:

“ Load the Spring application context and create the application’s beans so that I can test them together.

Conceptually:

```
@SpringBootTest  
↓  
Find @SpringBootConfiguration  
↓  
Load Spring ApplicationContext  
↓  
Create beans  
↓  
Dependency Injection  
↓  
Run test
```

Unlike Mockito unit testing, Spring is actually involved.

3. Unit Test vs `@SpringBootTest`

This distinction is extremely important in interviews.

Unit test

```
@ExtendWith(MockitoExtension.class)  
class PaymentServiceTest {  
  
    @Mock  
    PaymentRepository repository;  
  
    @InjectMocks  
    PaymentService service;  
}
```

Spring doesn’t start.

```
Test  
↓  
PaymentService  
↓  
Mock Repository
```

Fast.

Integration test

```
@SpringBootTest  
class PaymentServiceIntegrationTest {  
  
    @Autowired  
    PaymentService service;  
}
```

Spring starts the application context.

```
Test
↓
Spring ApplicationContext
↓
PaymentService
↓
Repository
↓
Database
```

Slower, but tests real Spring integration.

4. Simple Example

Suppose we have:

```
@Service
public class PaymentService {

    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }

    public Payment createPayment(Payment payment) {
        return repository.save(payment);
    }
}
```

Repository:

```
public interface PaymentRepository
    extends JpaRepository<Payment, Long> {
}
```

Integration test:

```
@SpringBootTest
class PaymentServiceIntegrationTest {

    @Autowired
    private PaymentService paymentService;

    @Autowired
    private PaymentRepository paymentRepository;

    @Test
    void shouldCreatePayment() {

        Payment payment = new Payment();
        payment.setAmount(BigDecimal.valueOf(1000));

        Payment saved = paymentService.createPayment(payment);

        assertNotNull(saved.getId());
        assertEquals(
            BigDecimal.valueOf(1000),
            saved.getAmount()
        );
    }
}
```

Here you're testing actual Spring-managed objects.

There is no:

```
@Mock
PaymentRepository repository;
```

because you want the real repository.

5. What exactly is being tested?

Suppose your application is:

```
PaymentController
  ↓
PaymentService
  ↓
PaymentRepository
  ↓
MySQL
```

With a suitable integration-test setup, `@SpringBootTest` can verify:

```
Spring Boot
  ↓
Controller bean created
  ↓
Service bean created
  ↓
Repository bean created
  ↓
Database connection
  ↓
SQL
  ↓
Result
```

This catches issues such as:

```
@Autowired
private PaymentService paymentService;
```

failing because the service isn't configured correctly.

A Mockito unit test would never discover that.

6. `@SpringBootTest` starts the entire application?

Not necessarily the HTTP server.

This is an important interview distinction.

By default:

```
@SpringBootTest
```

uses:

```
webEnvironment = WebEnvironment.MOCK
```

So Spring loads the web application context if appropriate, but it doesn't start an actual HTTP server.

You can explicitly choose the web environment.

7. webEnvironment

There are four important modes.

1. MOCK

```
@SpringBootTest(  
    webEnvironment = SpringBootTest.WebEnvironment.MOCK  
)
```

Default.

No real HTTP server.

Usually used with:

```
MockMvc
```

2. RANDOM_PORT

```
@SpringBootTest(  
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT  
)
```

Starts the actual embedded server on a random available port.

For example:

```
localhost:54321
```

You can test the application through actual HTTP.

3. DEFINED_PORT

```
@SpringBootTest(  
    webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT  
)
```

Uses the configured server port.

For example:

```
server.port=8080
```

Then:

```
localhost:8080
```

4. NONE

```
@SpringBootTest(  
    webEnvironment = SpringBootTest.WebEnvironment.NONE  
)
```

No web environment.

Useful when you're testing backend/service components without HTTP.

8. @SpringBootTest + MockMvc

This is a very common interview combination.

```
@SpringBootTest
@AutoConfigureMockMvc
class PaymentControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void shouldCreatePayment() throws Exception {

        mockMvc.perform(
            post("/payments")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                    {
                        "amount": 1000
                    }
                """)
        )
        .andExpect(status().isCreated());
    }
}
```

The request goes through:

```
MockMvc
↓
Controller
↓
Service
↓
Repository
```

but there is no real network connection.

This is often called an **integration test of the web layer**, depending on exactly what is included in the test configuration.

9. @WebMvcTest VS @SpringBootTest

This is a very common interview question.

@WebMvcTest

```
@WebMvcTest(PaymentController.class)
class PaymentControllerTest {
}
```

Loads primarily the MVC/web layer.

Typically:

```
Controller
MVC infrastructure
JSON serialization
Validation
```

Filters

Service dependencies are usually mocked.

Example:

```
@MockBean
private PaymentService paymentService;
```

or, in newer Spring versions, `@MockitoBean`.

@SpringBootTest

```
@SpringBootTest
class PaymentIntegrationTest {
}
```

Loads the broader application context.

Potentially:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

Interview comparison

	@WebMvcTest	@SpringBootTest
Scope	Web/MVC layer	Broad application context
Spring context	Partial	Full/broad
Service	Usually mocked	Can be real
Repository	Usually not loaded	Can be real
DB	No	Can be
Speed	Faster	Slower
Purpose	Controller testing	Integration testing

10. @SpringBootTest + Real Database

This is where integration testing becomes much more meaningful.

Suppose:

```
spring.datasource.url=jdbc:mysql://localhost:3306/payment_test
spring.datasource.username=test
spring.datasource.password=test
```

Then:

```
@SpringBootTest
class PaymentIntegrationTest {

    @Autowired
    private PaymentRepository repository;
```

```

@Test
void shouldSavePayment() {

    Payment payment = new Payment();

    payment.setAmount(
        BigDecimal.valueOf(500)
    );

    Payment result = repository.save(payment);

    assertNotNull(result.getId());
}

```

This tests:

```

Spring Data JPA
  ↓
Hibernate
  ↓
JDBC
  ↓
MySQL

```

Now you're testing real integration with the database.

11. But don't use your production database

⚠ COMMON TRAP

Absolutely not.

Integration tests should use an isolated test database.

Common approaches:

Option 1 — H2

```

Test
  ↓
H2

```

Easy, but H2 isn't always behaviorally identical to MySQL/PostgreSQL.

Option 2 — Testcontainers

This is much more realistic.

```

Integration Test
  ↓
Testcontainers
  ↓
Docker container
  ↓
MySQL/PostgreSQL

```

Example:

```

@Testcontainers
@SpringBootTest
class PaymentIntegrationTest {

```

```

@Container
static MySQLContainer<?> mysql =
    new MySQLContainer<>("mysql:8.0");
}

```

Testcontainers is a very useful technology to know for modern backend interviews.

12. `@Transactional` in Integration Tests

You will frequently see:

```

@SpringBootTest
@Transactional
class PaymentIntegrationTest {
}

```

For example:

```

@Test
void shouldSavePayment() {

    Payment payment = new Payment();
    payment.setAmount(BigDecimal.valueOf(1000));

    repository.save(payment);

    assertEquals(1, repository.count());
}

```

With Spring's test transaction management, the transaction can be rolled back after the test.

Conceptually:

```

BEGIN TRANSACTION
  ↓
Test executes
  ↓
Assertions
  ↓
ROLLBACK

```

This helps keep the database clean between tests.

⚠ COMMON TRAP

Important caveat

Don't blindly assume every integration test rolls back.

Transaction behavior depends on how the test is configured and where the transaction is started. Also, operations performed in separate transactions or through external systems may not be undone by the test transaction.

13. Testing the Full HTTP Flow

For a realistic API integration test:

```

@SpringBootTest(
    webEnvironment =

```

```

        SpringApplication.run(SpringBootTest.WebEnvironment.RANDOM_PORT, args);
    }

    @SpringBootTest
    class PaymentApiIntegrationTest {

        @Autowired
        private TestRestTemplate restTemplate;

        @Test
        void shouldCreatePayment() {

            PaymentRequest request =
                new PaymentRequest(
                    BigDecimal.valueOf(1000)
                );

            ResponseEntity<PaymentResponse> response =
                restTemplate.postForEntity(
                    "/payments",
                    request,
                    PaymentResponse.class
                );

            assertEquals(
                HttpStatus.CREATED,
                response.getStatusCode()
            );
        }
    }
}

```

Flow:

```

TestRestTemplate
  ↓
HTTP
  ↓
Embedded Tomcat
  ↓
Spring MVC
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database

```

This is closer to testing the application as a real client would use it.

14. Do we mock anything in `@SpringBootTest`?

Yes.

This is where people sometimes misunderstand integration testing.

Integration testing does **not** mean:

“Never use mocks.”

You might have:

```

Application
  ↓
PaymentService
  ↓
PaymentRepository

```

```
↓
Database
PaymentService
↓
Hydra external API
```

You may want the database to be real but Hydra to be mocked.

Why?

Because your integration test shouldn't depend on:

- Internet
- External API availability
- External API data
- API credentials
- External service rate limits

So you can do:

```
Real:
Controller
Service
Repository
Database

Mock:
Hydra
```

This is often a very practical integration-test boundary.

15. `@MockBean` / `@MockitoBean`

Historically:

```
@MockBean
private HydraClient hydraClient;
```

This tells Spring to put a Mockito mock into the application context.

Then:

```
when(hydraClient.processPayment(any()))
    .thenReturn(successResponse);
```

Your actual service runs:

```
PaymentService
↓
Mock HydraClient
```

while the rest of the application can remain real.

Current Spring note

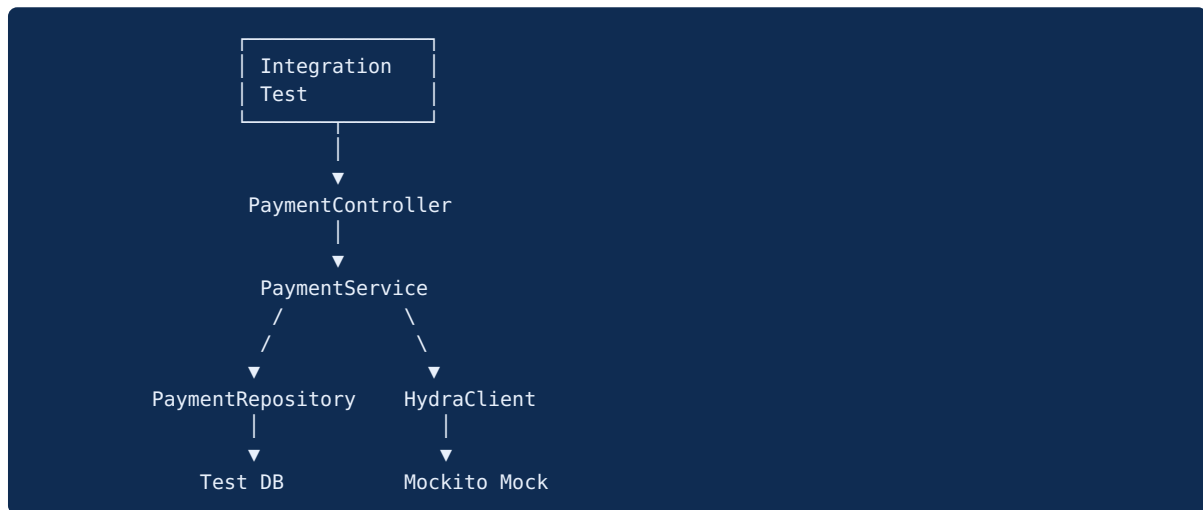
In newer Spring Framework/Spring Boot versions, `@MockBean` has been deprecated in favor of:

```
@MockitoBean
```

So when working in a current codebase, check the Spring Boot version and use the appropriate annotation.

16. Integration Test Architecture

A realistic payment integration test might look like:



This is a **very useful interview example** because it demonstrates that you understand test boundaries.

17. `@SpringBootTest` is expensive

One major disadvantage:

```
JUnit test
↓
Start Spring
↓
Create beans
↓
Configure infrastructure
↓
Run test
```

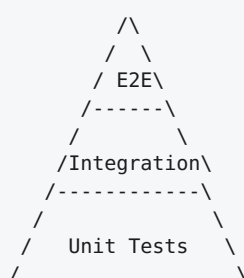
This is much slower than:

```
Mockito
↓
Create a few objects
↓
Run test
```

Therefore, don't write every test as:

```
@SpringBootTest
```

A good test pyramid looks roughly like:



Many fast unit tests, fewer integration tests, very few expensive end-to-end tests.

18. Spring Context Caching

An important advanced point.

Suppose you have:

```
@SpringBootTest
class PaymentTest {
}
```

and:

```
@SpringBootTest
class OrderTest {
}
```

Spring can **cache the ApplicationContext** between tests when their configurations are compatible.

So it doesn't necessarily start a completely new Spring application for every test method.

This is one reason test suites can be optimized.

However, changing test configuration, profiles, mocks, properties, etc. can cause different contexts to be created.

19. Profiles for Integration Testing

Very common:

```
@SpringBootTest
@ActiveProfiles("test")
class PaymentIntegrationTest {
}
```

Then:

```
application.yml
application-test.yml
```

Example:

```
# application-test.yml

spring:
  datasource:
    url: jdbc:h2:mem:testdb
```

Your test environment becomes isolated from production configuration.

20. What should you test with `@SpringBootTest`?

Good candidates:

1. Bean wiring

Controller → Service → Repository

2. Database interaction

Repository → Hibernate → DB

3. Transaction behavior

Service
↓
Multiple DB operations
↓
Commit/Rollback

4. Serialization

JSON → DTO
DTO → JSON

5. Validation

HTTP Request
↓
Bean Validation
↓
Controller

6. Configuration

application-test.yml
↓
Spring beans

7. Multiple components working together

For example:

Controller
↓
Service
↓
Repository

21. What NOT to do

⚠ COMMON TRAP

Don't write:

```
@SpringBootTest
```

for every tiny method.

For example:

```
public BigDecimal calculateFee(BigDecimal amount) {  
    return amount.multiply(  
        BigDecimal.valueOf(0.02)  
    );  
}
```

You don't need Spring to test that.

Use:

```
@Test
void shouldCalculateFee() {
    ...
}
```

or a Mockito/unit test if dependencies exist.

22. Interview Question: Is `@SpringBootTest` an End-to-End Test?

INTERVIEW SCENARIO

No.

Strong answer:

“`@SpringBootTest` loads the Spring application context and is primarily used for integration testing. Depending on its configuration, it can test multiple application layers together, and with `RANDOM_PORT` it can exercise the application through real HTTP. However, that doesn't automatically make it a complete end-to-end test because external systems such as payment providers, identity providers, and downstream services may still be mocked or replaced.”

That's a much better answer than:

“`@SpringBootTest` is for E2E testing.”

23. Interview Question: Why Not Use `@SpringBootTest` Everywhere?

INTERVIEW SCENARIO

Answer:

“`@SpringBootTest` starts a substantial Spring application context, so tests are slower and more expensive than isolated unit tests. I use unit tests for business logic and integration tests when I need to verify Spring wiring, persistence, transactions, serialization, or interactions between multiple components.”

24. `@SpringBootTest` VS `@DataJpaTest`

Another important comparison:

Annotation	Purpose

@SpringBootTest	Broad application integration
@WebMvcTest	MVC/controller layer
@DataJpaTest	JPA/repository layer
@JsonTest	JSON serialization/deserialization
@SpringBootTest + @AutoConfigureMockMvc	Full Spring context + MockMvc

For example:

```
@DataJpaTest
class PaymentRepositoryTest {
}
```

is more focused than:

```
@SpringBootTest
class PaymentRepositoryTest {
}
```

If you're testing only repository behavior, @DataJpaTest is generally the better choice.

25. Unit → Integration → E2E

Think about the levels like this:

Unit

```
PaymentService
  ↓
Mockito mocks
```

Question:

“Does this class behave correctly?”

Integration

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Test DB
```

Question:

“Do these components work together correctly?”

E2E

```
Real client
  ↓
API Gateway
  ↓
```

```
Payment Service
  ↓
Database
  ↓
Real/controlled downstream services
```

Question:

“

“Does the complete business workflow work in an environment close to production?”

26. BPS-specific application

Given your BPS architecture, a sensible integration-test boundary could be:

```
BPS API
  ↓
Controller
  ↓
Payment Service
  ↓
Vendor Strategy
  ↓
Repository / BHS interaction
```

while mocking an external provider such as:

Hydra

or:

Chase

if those are external dependencies in the particular flow you’re testing.

For example:

```
POST /payments
  ↓
BPS Controller      REAL
  ↓
BPS Service         REAL
  ↓
Vendor Strategy     REAL
  ↓
Hydra client        MOCK
  ↓
Database            REAL TEST DB
```

This would test considerably more than a Mockito unit test while remaining deterministic.

When you inspect your BPS codebase, look for:

1. `@SpringBootTest`
2. `@WebMvcTest`
3. `@DataJpaTest`
4. `@ActiveProfiles("test")`
5. `@AutoConfigureMockMvc`
6. `@MockBean` / `@MockitoBean`
7. `@Transactional` on tests
8. Testcontainers
9. H2/test database configuration

10. How Hydra/Chase calls are mocked
 11. Whether integration tests execute in CI/CD
 12. Whether external API contracts are tested separately
-

Interview Cheat Sheet

INTERVIEW SCENARIO

If the interviewer asks “**Explain @SpringBootTest**”, give this:

“

“@SpringBootTest is used for Spring Boot integration testing. It loads the application context and creates Spring-managed beans, allowing us to test multiple layers together rather than testing a class in isolation. Depending on the web environment, we can test through MockMvc or start an actual embedded server using a random port. We typically combine it with a test profile and an isolated database, and mock external systems when necessary. Because loading the Spring context is relatively expensive, I wouldn't use it for every unit test.”

That is the level of answer I would expect from a **3+ year Spring Boot developer**.